

VidStyler: AI-Powered Image and Video Editing

ABSTRACT

The proliferation of digital media has led to an increasing demand for powerful yet accessible image and video editing tools. Conventional editing software often presents a steep learning curve, hindering casual users and content creators. The VidStyler project addresses this challenge by creating an integrated, AI-powered suite that simplifies complicated editing tasks, thereby democratizing advanced media editing capabilities. The project's goal is to make advanced media editing more accessible, allowing users to improve their creative output with little to no technical expertise.

VidStyler offers three core functionalities: Neural Style Transfer (NST), Video Stabilization, and Object Removal. The NST feature uses a VGG16 network for feature extraction and an L-BFGS optimization to transfer the artistic style of one image onto another. Video stabilization is achieved using keypoint detection algorithms like GFTT, SIFT, and ORB via the `vidstab` library to smooth camera motion. Object Removal uses intelligent inpainting with either traditional OpenCV methods or the advanced TensorFlow-based DeepFill model to eliminate unwanted elements from images. The entire suite is presented through a user-friendly web interface built with Gradio, with separate tabs for each function and interactive controls for adjusting parameters. The report details the project's architecture, implementation, and results, demonstrating a practical application of AI in multimedia editing.

Contents

Chapter	Page No.
Abstract	i
Contents	ii
List of Figures	iv
List of Tables	v
Chapter 1: Introduction	1
1.1 Existing Work	1
1.2 The Gap and Motivation	2
1.3 VidStyler: Enhancements and Objectives	2
Chapter 2: Literature Survey	4
2.1 Image Style Transfer	4

Chapter	Page No.
2.2 Video Stabilization	5
2.3 Object Removal (Inpainting)	6
Chapter 3: System Analysis	8
3.1 Proposed System and Functional Requirements	8
3.2 Hardware Requirements	9
3.3 Software Requirements	10
3.4 Non-Functional Requirements	11
Chapter 4: System Design	12
4.1 Architectural Design	12
4.2 Module Breakdown	13
4.3 Use Case Model	14
4.4 Sequence Diagrams	15
Chapter 5: Implementation	17
5.1 User Interface (app/app.py)	17
5.2 Neural Style Transfer Module	18
5.3 Video Stabilization Module	19
5.4 Object Removal Module	19
Chapter 6: Algorithms	21
6.1 Neural Style Transfer (NST)	21
6.2 Video Stabilization	22
6.3 Object Removal (Inpainting)	23
Chapter 7: Testing and Results	24
7.1 Testing Approach	24
7.2 Results and Discussion	25
Chapter 8: Conclusion and Future Work	28
8.1 Conclusion	28
8.2 Future Work	28
Chapter 9: References	30

List of Figures

Figure No.	Figure Name	Page No.
4.1	High-Level System Architecture of VidStyler	12
4.2	Detailed Modular Architecture of VidStyler	13
4.3	Use Case Diagram for VidStyler	14
4.4	Sequence Diagram for Neural Style Transfer	15
4.5	Sequence Diagram for Video Stabilization	15
4.6	Sequence Diagram for Object Removal	16
7.1	Neural Style Transfer UI and Example Output	25
7.2	Video Stabilization UI and Example Output (with Plots)	26
7.3	Object Removal UI and Example Output	27

List of Tables

Table No.	Table Name	Page No.
3.1	Hardware Requirements	9
3.2	Software Requirements	10

Chapter 1: Introduction

1.1 Existing Work

The field of digital media editing has seen significant advancements, driven by the need for more efficient and powerful tools. Traditional editing software, such as Adobe Photoshop and Premiere Pro, offers extensive manual controls but often requires specialized technical skills and significant time investment. In response to the growing demand for more accessible editing, various tools have emerged, often focusing on specific tasks.

Neural Style Transfer (NST) was pioneered by Gatys et al. (2015), who demonstrated that the artistic style of one image could be separated and applied to the content of another using a pre-trained Convolutional Neural Network (CNN) like VGG16. This optimization-based approach, which minimizes content and style losses, laid the groundwork for numerous applications. Subsequent work, like that of Johnson et al. (2016), introduced feed-forward networks for faster, real-time style transfer, but the original iterative optimization method remains a powerful technique for achieving high-quality results.

Video stabilization has a rich history, with methods ranging from simple frame-by-frame transformations to complex motion trajectory analysis. Early techniques relied on motion vectors and optical flow, while more modern approaches utilize keypoint tracking algorithms such as GFTT, SIFT, and ORB to estimate inter-frame motion. Libraries like `vidstab` have encapsulated these methods, providing a robust framework for smoothing camera paths and compensating for unwanted jitters.

Object removal, also known as inpainting, has evolved from traditional patch-based methods to sophisticated deep learning techniques. Classical methods like those based on Navier-Stokes and exemplar-based approaches are effective for small, simple regions but struggle with complex textures and large occlusions. The advent of deep learning has revolutionized this field. Models like DeepFill, which use gated convolutions and contextual attention mechanisms, have demonstrated remarkable capabilities in generating semantically coherent and visually plausible content to fill arbitrary holes in images.

Despite these individual advancements, a common challenge persists: these tools are often disparate. Users must navigate different platforms, each with its own interface and workflow, to perform various editing tasks. The existing landscape is characterized by powerful, specialized tools for experts and simplified, but often limited, tools for a general audience. The primary gap lies in the lack of an integrated, accessible, and powerful suite that democratizes these advanced AI-powered functionalities.

1.2 The Gap and Motivation

The development of VidStyler is motivated by several key factors that highlight a significant gap in the current market for digital media editing tools. The explosion of user-generated content across social media, vlogging, and digital marketing has created an immense demand for high-quality, engaging media. However, many content creators, individuals, and small businesses lack the technical expertise and financial resources to use professional-grade software effectively. This creates a barrier to entry for producing polished visual content.

While advancements in AI have provided powerful solutions for tasks like style transfer, stabilization, and object removal, these tools often remain inaccessible. They are frequently presented as research projects with complex command-line interfaces or require extensive knowledge of hyperparameters. A user needing to stabilize a video and then remove an object would typically have to use two different applications, leading to a disjointed and inefficient workflow.

The core problem VidStyler addresses is the discrepancy between the power of cutting-edge AI and the usability of a cohesive, integrated platform. There is a pressing need for a unified solution that translates complex algorithms into user-friendly features. VidStyler aims to bridge this gap by offering a single, intuitive interface for multiple high-impact editing functionalities. This approach not only streamlines the creative process but also makes

sophisticated media manipulation techniques approachable for a much wider audience, empowering users to achieve professional-looking results without being AI experts.

1.3 VidStyler: Enhancements and Objectives

VidStyler is designed to fill the identified gaps by providing an integrated, AI-powered image and video editing suite that is both powerful and accessible. The project's primary objective is to develop a single platform that consolidates three high-demand functionalities:

1. **Neural Style Transfer (NST):** We implement a robust NST module that uses a pre-trained VGG16 network for feature extraction and an L-BFGS optimization process to minimize content and style losses. This module allows users to fine-tune the output by adjusting parameters such as style weight, content weight, and iteration count through a simple interface.
2. **Video Stabilization:** The video stabilization module leverages the `vidstab` library, supporting multiple keypoint detection methods including GFTT, SIFT, and ORB. Users can customize the smoothing radius and border handling techniques, effectively reducing camera shake and producing smoother, more professional-looking footage.
3. **Intelligent Object Removal:** This feature enables users to seamlessly remove unwanted elements from images. The system offers an interactive masking tool and a choice between a fast OpenCV-based inpainting method for quick results and a more advanced, TensorFlow-based DeepFill model for high-quality, context-aware inpainting of complex regions.

The entire suite is housed within an intuitive, Gradio-based web interface. This user-friendly UI abstracts away the underlying technical complexity, providing clear tabs for each function and interactive controls. The system is built with a modular and maintainable architecture, with separate directories for the UI, core processing logic for each feature, and utility functions. This design ensures that VidStyler is not only a functional tool but also a well-structured and scalable project. By offering a unified platform with robust, AI-driven features and an accessible interface, VidStyler addresses the key challenges of accessibility, computational intensity, and tool fragmentation, ultimately empowering a broader audience of content creators.

Chapter 2: Literature Survey

This chapter reviews existing research and techniques relevant to the core functionalities of VidStyler: Image Style Transfer, Video Stabilization, and Object Removal (Inpainting).

2.1 Image Style Transfer

Image Style Transfer aims to render the content of one image in the artistic style of another.

- **Early Non-Photorealistic Rendering (NPR) Techniques:** Before deep learning, methods for artistic rendering included stroke-based rendering, which simulated brush

strokes, and image filtering techniques to achieve effects like watercolor or impressionism. Texture synthesis and image analogies (Hertzmann et al., 2001) were also explored to transfer visual appearance. These methods often required significant parameter tuning or manual intervention.

- **Neural Style Transfer (Gatys et al., 2015):** The seminal work "A Neural Algorithm of Artistic Style" by Gatys, Ecker, and Bethge revolutionized the field. They demonstrated that deep Convolutional Neural Networks (CNNs), specifically VGG networks pre-trained on ImageNet, could separate and recombine the content and style of images.
 - **Content Representation:** Captured by the feature responses in the higher layers of a CNN.
 - **Style Representation:** Captured by the correlations between feature responses across different channels, represented by Gram matrices, typically extracted from multiple layers of the CNN.
 - **Process:** An optimization process (often using L-BFGS) iteratively modifies an initial image to minimize a weighted sum of content loss and style loss. This method produces high-quality results but is computationally intensive and slow.
- **Fast Neural Style Transfer (Johnson et al., 2016; Ulyanov et al., 2016):** To address the speed limitations of the optimization-based approach, researchers proposed training feed-forward neural networks. These networks are trained to transform any content image into a specific style. Once trained, applying the style is very fast (a single forward pass), but a separate network needs to be trained for each new style.
- **Arbitrary Style Transfer / Universal Style Transfer:** The next advancement aimed to allow fast style transfer using any arbitrary style image without retraining.
 - **AdaIN (Adaptive Instance Normalization) (Huang and Belongie, 2017):** Proposed aligning the mean and variance of content features with those of style features in the feature space, allowing for real-time arbitrary style transfer.
 - **WCT (Whitening and Coloring Transform) (Li et al., 2017):** Another approach that stylizes content features by matching their second-order statistics to those of the style features.
- **GAN-based Style Transfer:** Generative Adversarial Networks (GANs) have also been applied, for example, in CycleGAN (Zhu et al., 2017) for unpaired image-to-image translation.

VidStyler implements the original optimization-based approach (Gatys et al.) for its Neural Style Transfer feature, prioritizing quality and flexibility in controlling the style/content balance.

2.2 Video Stabilization

Video stabilization aims to remove undesirable camera shakes and jitters from video sequences.

- **2D Electronic Image Stabilization (EIS):** These are common in consumer cameras and smartphones. They typically estimate global motion (e.g., affine or homography) between frames and then smooth this motion.
 - **Feature-based methods:** Track salient feature points (e.g., SIFT, SURF, ORB, GFTT) across frames to estimate the camera's motion.
 - **Motion Smoothing:** The estimated camera trajectory is smoothed using filters like moving average, Gaussian, or Kalman filters.
 - **Image Warping:** Frames are warped according to the difference between the original and smoothed trajectories. The `vidstab` library, used in VidStyler, largely follows this paradigm.
- **2.5D and 3D Video Stabilization:**
 - **2.5D:** Some methods attempt to model parallax by dividing the scene into layers or using depth information.
 - **3D Methods:** These methods reconstruct the 3D camera path and scene geometry (Structure from Motion - SfM). They can provide very high-quality stabilization but are computationally expensive.
- **Deep Learning for Video Stabilization:** More recent approaches use deep learning.
 - **Learning to Predict Transformations:** CNNs can be trained to predict stabilizing transformations directly from pairs of frames or short video clips.
 - **StabNet (Wang et al., 2018):** An example of a deep learning approach that estimates homographies for stabilization.

VidStyler uses the `vidstab` library, which primarily relies on 2D feature-based motion estimation and trajectory smoothing, offering a good balance between effectiveness and computational feasibility.

2.3 Object Removal (Inpainting)

Object removal, or inpainting, is the process of filling in missing or unwanted regions in an image in a visually plausible manner.

- **Traditional Inpainting Methods:**
 - **Diffusion-based (Bertalmio et al., 2000; Telea, 2004):** These methods propagate information from the boundary of the missing region inwards using techniques inspired by partial differential equations (PDEs). They are good for small regions but tend to blur larger areas. OpenCV implements these (e.g., `cv2.INPAINT_NS`, `cv2.INPAINT_TELEA`).
 - **Patch-based / Exemplar-based (Criminisi et al., 2004):** These methods search for similar patches in the known part of the image and copy them into the unknown region. They are better at reconstructing textures but can be slow and may produce repetitive patterns.
- **Deep Learning-based Inpainting:** These methods have shown significantly superior results, especially for large missing regions.

- **Context Encoders (Pathak et al., 2016):** An early deep learning approach using an encoder-decoder architecture with an adversarial loss.
- **DeepFill & DeepFill v2 (Yu et al., 2018, 2019):** A prominent architecture that introduced gated convolutions and contextual attention mechanisms to handle irregular holes and borrow feature information from distant spatial locations. This is the ML-based method offered in VidStyler.
- **GAN-based methods:** Many deep learning inpainting methods leverage Generative Adversarial Networks (GANs) to produce sharper and more realistic results.

VidStyler provides both traditional OpenCV-based inpainting (fast, for simple cases) and the more advanced DeepFill method (TensorFlow-based, better for complex scenes).

Chapter 3: System Analysis

This chapter details the analysis of the VidStyler system, covering its functional requirements, hardware and software prerequisites, and other non-functional considerations.

3.1 Proposed System and Functional Requirements

VidStyler is an AI-powered image and video editing suite designed to provide users with advanced editing capabilities through an intuitive interface. The system integrates three core functionalities:

1. Neural Style Transfer (NST):

- **FR1.1:** Allow users to upload a content image and a style image.
- **FR1.2:** Apply the artistic style of the style image to the content image.
- **FR1.3:** Utilize a VGG16 pre-trained model for feature extraction.
- **FR1.4:** Employ an L-BFGS optimization algorithm to generate the stylized image.
- **FR1.5:** Allow users to adjust parameters: Style Weight, Content Weight, and Iterations.
- **FR1.6:** Display the resulting stylized image and provide progress feedback.

2. Video Stabilization:

- **FR2.1:** Allow users to upload a video file.
- **FR2.2:** Reduce camera shake and unwanted motion.
- **FR2.3:** Support multiple keypoint detection methods (e.g., GFTT, SIFT, ORB).
- **FR2.4:** Allow users to customize the smoothing radius and border handling.
- **FR2.5:** Display trajectory plots for analysis.
- **FR2.6:** Output the stabilized video file and provide status updates.

3. Object Removal (Inpainting):

- **FR3.1:** Allow users to upload an image.

- **FR3.2:** Provide an interactive image editor where users can draw a mask over unwanted objects.
- **FR3.3:** Offer multiple inpainting methods: OpenCV-based (Navier-Stokes, Telea) and DeepFill (ML-based).
- **FR3.4:** Display the image with the selected objects removed.
- **FR3.5:** Provide a "Reset Image" option to clear user drawings.

3.2 Hardware Requirements

To effectively run VidStyler, especially its computationally intensive AI modules, the following hardware specifications are suggested:

Table 3.1 Hardware Requirements

Component	Minimum Specification	Recommended Specification	Notes
Processor (CPU)	Multi-core (e.g., Intel i5 4th Gen equivalent)	Modern Multi-core (e.g., Intel i5 11th Gen+, AMD Ryzen 5 3000 series+)	Faster CPU improves general responsiveness and non-GPU tasks.
RAM	8 GB	16 GB or more	More RAM is crucial for handling larger images/videos and ML models.
Storage	50 GB free space (HDD)	100 GB free space (SSD)	SSD significantly improves model loading times and file operations.
GPU	Not strictly required (CPU fallback)	NVIDIA CUDA-enabled GPU (e.g., GTX 1650+, RTX series) with 4GB+ VRAM	Highly recommended for NST and DeepFill for acceptable performance.

3.3 Software Requirements

VidStyler relies on several software components and libraries. The development and execution environment should meet these requirements:

Table 3.2 Software Requirements

Category	Component/Library	Version / Details	Purpose
Operating System	Windows, macOS, Linux	Windows 10/11, recent macOS, common Linux distributions	Platform for running the application.

Category	Component/Library	Version / Details	Purpose
Python	Python	3.8+	Core programming language.
Deep Learning	PyTorch	1.8+ (with CUDA support if GPU is used)	For Neural Style Transfer (VGG16, optimization).
	TensorFlow	1.x or 2.x with compat.v1	For DeepFill object removal.
Computer Vision	OpenCV (cv2)	4.5+	Image/video processing, traditional inpainting.
UI Framework	Gradio	2.0+	For building the web-based user interface.
Video Processing	vidstab	Latest stable version	For video stabilization.
Core Libraries	NumPy, Pillow, Matplotlib	Latest stable versions	Numerical operations, image I/O, and plotting.
Web Browser	Chrome, Firefox, Edge, Safari	Modern versions	For accessing the Gradio UI.

3.4 Non-Functional Requirements

- **NFR1 (Usability):** The application shall be intuitive and easy to use, even for users with limited technical expertise.
- **NFR2 (Performance):** The UI should be responsive. Processing times should be reasonable, with progress updates for computationally intensive tasks like NST and DeepFill.
- **NFR3 (Reliability):** The application should handle common errors gracefully (e.g., invalid file uploads) and provide informative error messages.
- **NFR4 (Maintainability):** The codebase should be well-structured, modular, and adequately commented to facilitate future updates.
- **NFR5 (Modularity):** The three main functionalities should be implemented as distinct modules, allowing for independent development and testing.

Chapter 4: System Design

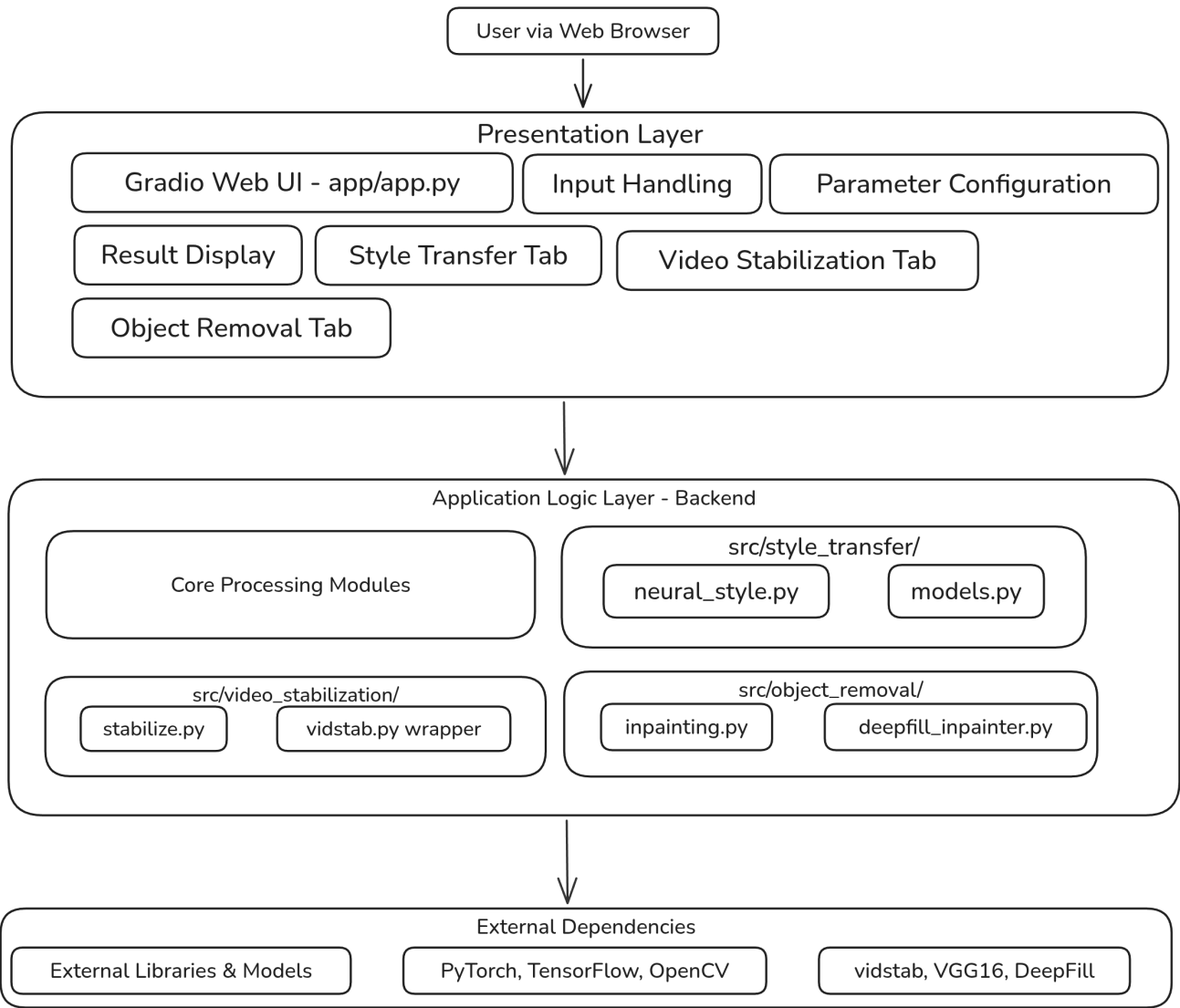
This chapter outlines the design of the VidStyler system, covering the high-level architecture, modular breakdown, and dynamic interactions within the system.

4.1 Architectural Design

VidStyler employs a modular, layered architecture to separate concerns and promote maintainability. The system is visualized as having two main layers:

1. **Presentation Layer (User Interface):** Handles user interaction, input gathering, and result display. It is implemented using **Gradio** and defined in `app/app.py`.
2. **Application Logic/Processing Layer (Backend):** Contains the core AI and image/video processing functionalities, located in the `src/` directory.

Interaction Flow: A user interacts with the Gradio UI, which calls functions in `app.py`. These functions then invoke the core logic in the `src/` modules. The results are returned to `app.py`, and the Gradio UI updates to display them.

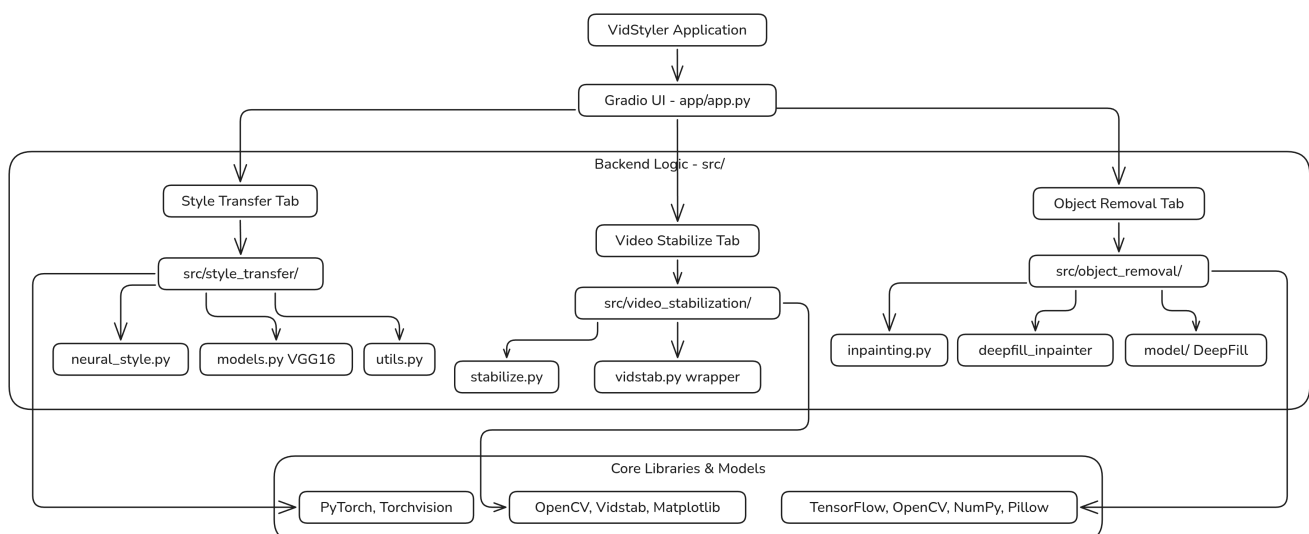


4.2 Module Breakdown

The system is broken down into the following key modules:

1. **`app/app.py` (UI Application Core):** Defines the Gradio interface structure (Tabs, Rows, Columns, Input/Output components) and connects user actions to backend functions.
2. **`src/style_transfer/` (Neural Style Transfer Module):**

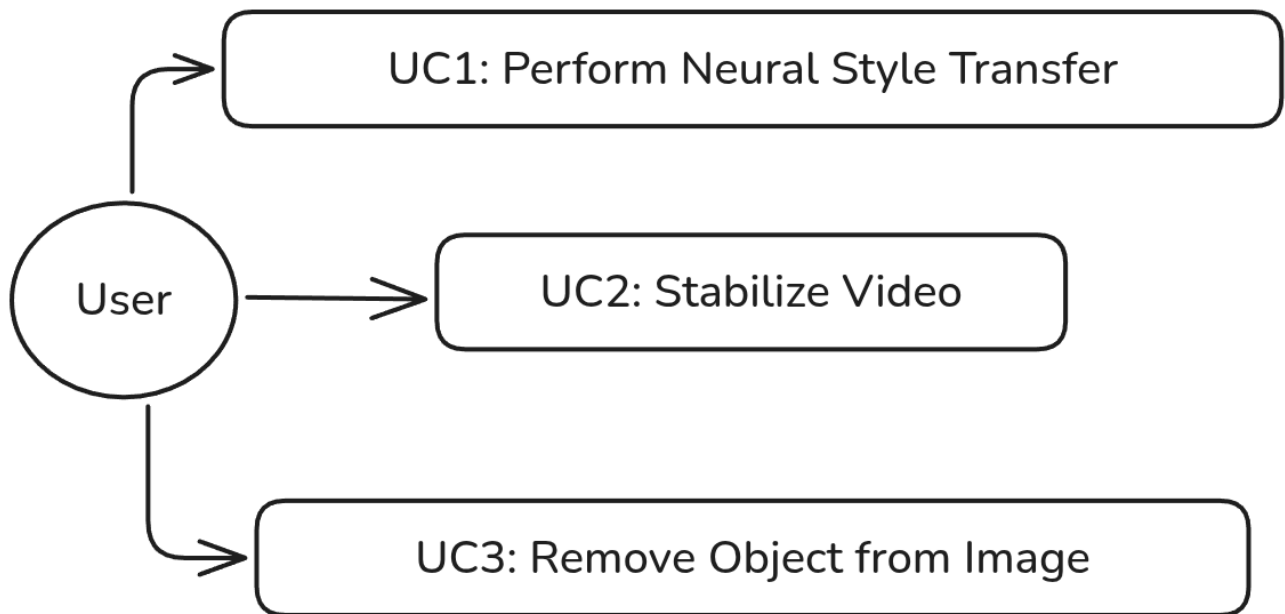
- `neural_style.py` : Contains the main logic for NST, including the L-BFGS optimization loop.
 - `models.py` : Defines the VGG16 network for feature extraction.
3. **src/video_stabilization/ (Video Stabilization Module):**
- `stabilize.py` : High-level function called by the UI to orchestrate stabilization.
 - `vidstab.py` : A custom wrapper around the external `vidstab` library.
4. **src/object_removal/ (Object Removal Module):**
- `inpainting.py` : Central script that selects the inpainting method (OpenCV or DeepFill).
 - `deepfill_inpainter.py` : Manages the loading and inference of the pre-trained DeepFill TensorFlow model.
 - `model/` : Contains the DeepFill model architecture definition.
5. **run.py (Application Launcher):** The entry point that initializes and launches the Gradio application.



4.3 Use Case Model

The use case model describes the system's functionality from the perspective of the **User**.

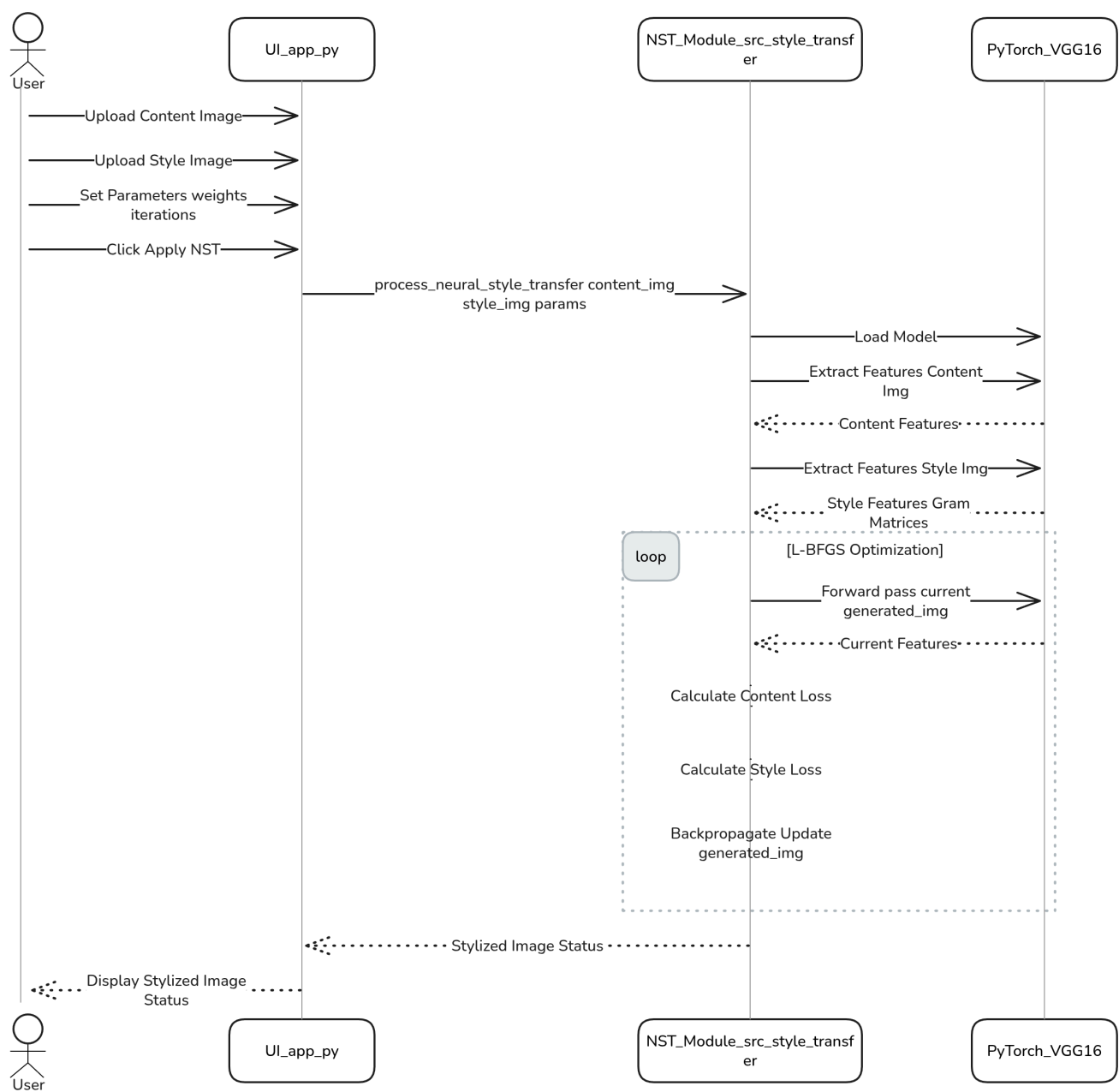
- **UC1: Perform Neural Style Transfer:** User uploads content and style images, adjusts parameters, and receives a stylized image.
- **UC2: Stabilize Video:** User uploads a video, selects stabilization parameters, and receives a stabilized video and optional analysis plots.
- **UC3: Remove Object from Image:** User uploads an image, draws a mask on an object, selects an inpainting method, and receives an image with the object removed.



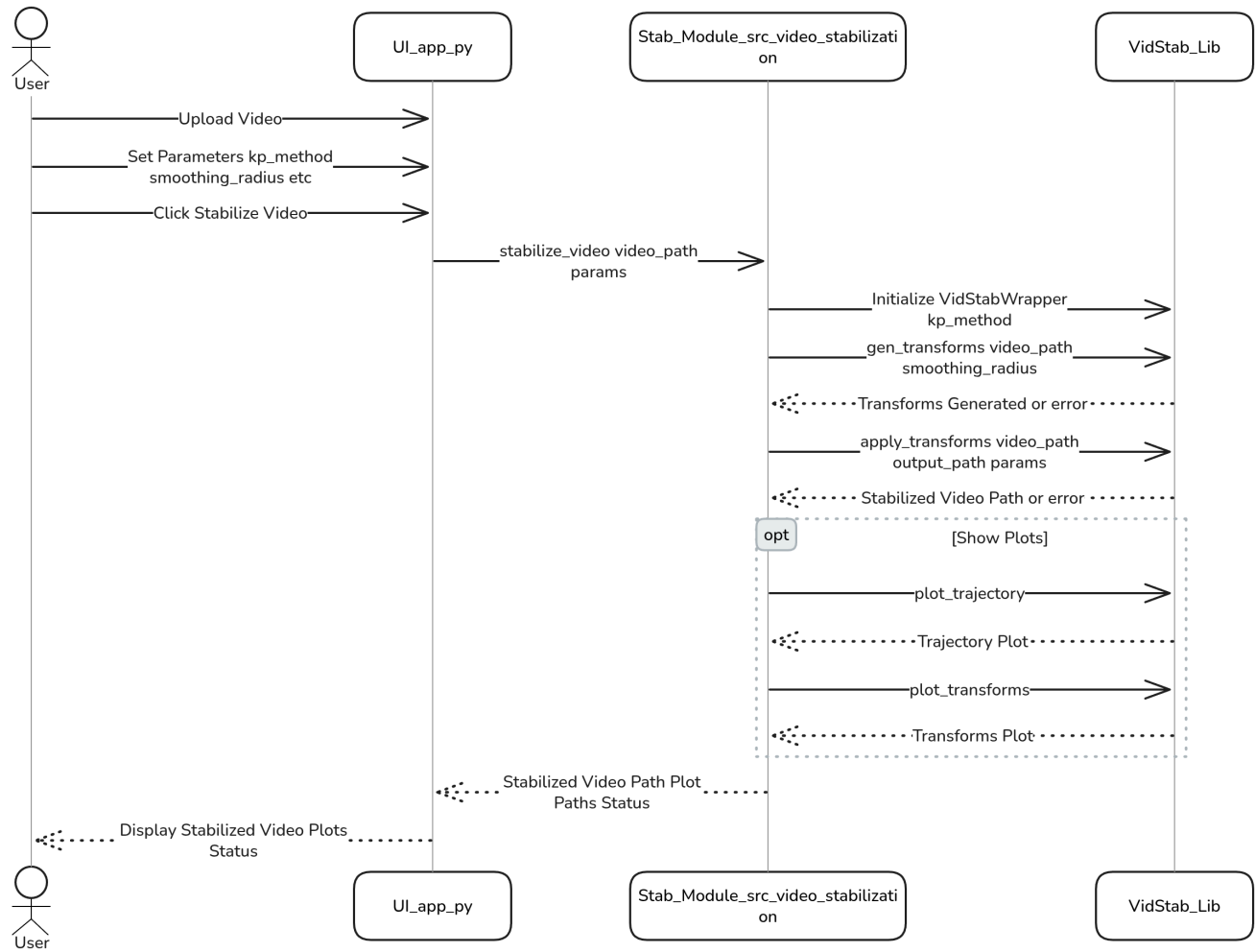
4.4 Sequence Diagrams

Sequence diagrams illustrate the time-ordered interactions between system components for each use case.

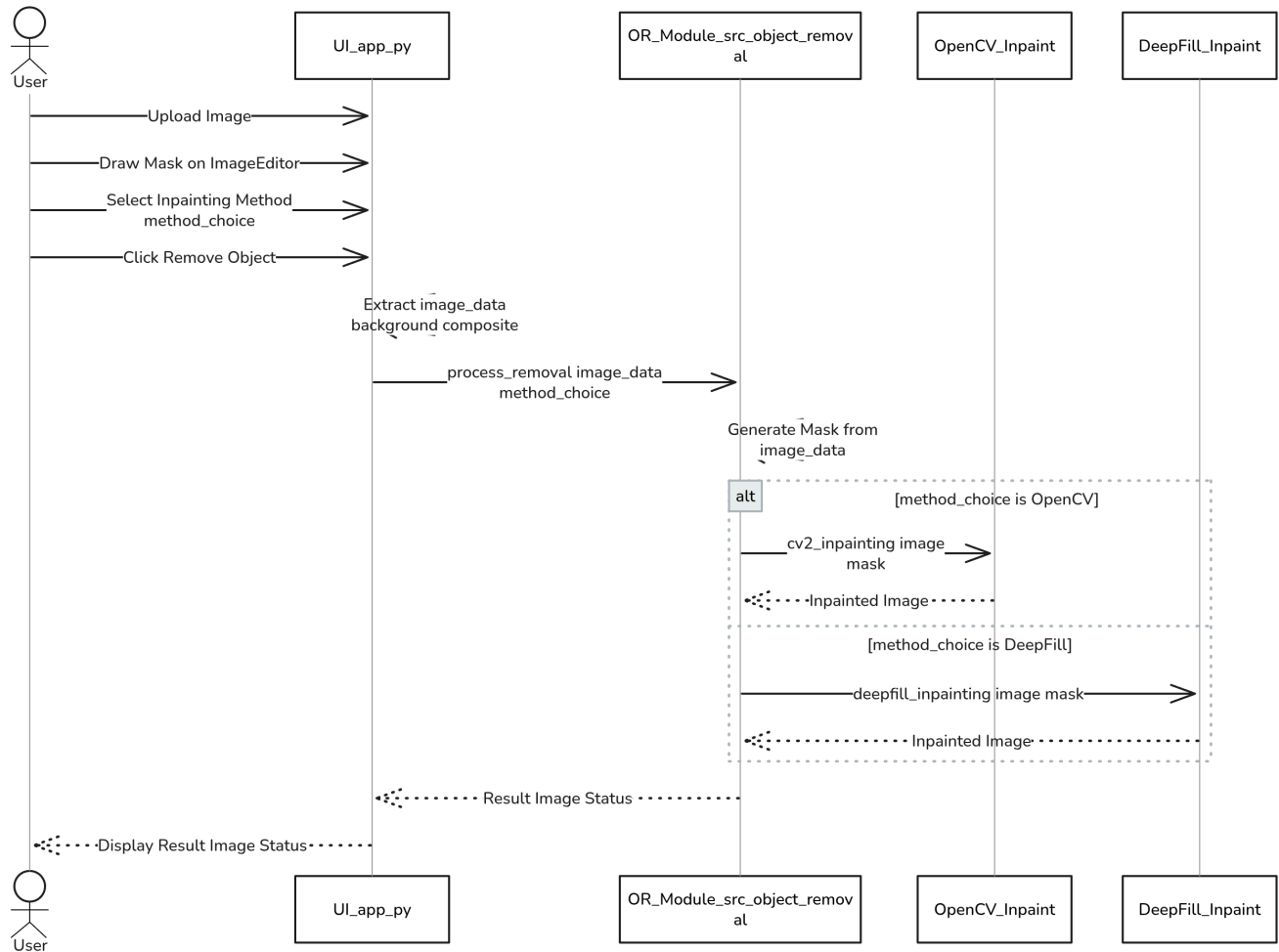
Neural Style Transfer Sequence



Video Stabilization Sequence



Object Removal Sequence



Chapter 5: Implementation

This chapter describes the implementation details of the VidStyler application, focusing on the key modules and how they realize the functionalities defined in the design.

5.1 User Interface (app/app.py)

The user interface is the primary point of interaction and is built using Gradio. The `app/app.py` file orchestrates the UI and connects frontend components to backend processing logic.

- **Structure:** A `gr.Blocks()` context defines the layout, with `gr.Tabs()` separating the three core functionalities: "Style Transfer," "Video Stabilization," and "Object Removal."
- **Style Transfer Tab:** Contains `gr.Image` inputs for content and style images, `gr.Slider` controls for weights and iterations, a `gr.Button` to trigger the process, and `gr.Image` and `gr.Textbox` for output and status.
- **Video Stabilization Tab:** Uses a `gr.Video` input, with `gr.Dropdown`, `gr.Slider`, and `gr.Radio` controls for parameters like keypoint method, smoothing radius, and border handling. Outputs include a `gr.Video` component for the result and `gr.Image` components for plots.

- **Object Removal Tab:** Features a `gr.ImageEditor` component, which allows users to upload an image and draw a mask directly. The backend logic computes a binary mask from the difference between the background and composite layers of the editor's output. `gr.Radio` buttons allow method selection, and a `gr.Button` initiates the removal.

5.2 Neural Style Transfer Module (`src/style_transfer/`)

This module implements the optimization-based NST.

- **neural_style.py** : The `neural_style_transfer()` function implements the core logic. It loads images as PyTorch tensors, initializes the VGG16 model, and sets up an `LBFGS` optimizer. Inside an optimization loop, a `closure` function calculates the total loss (a weighted sum of content and style loss) and performs the backward pass. The optimizer then updates the input tensor's pixels.
- **models.py** : Defines a `VGG16` class that wraps the pre-trained `torchvision` model, sliced to easily extract features from intermediate layers.
- **utils.py** : Contains helper functions, most notably `gram_matrix()`, which computes the style representation from feature maps.

5.3 Video Stabilization Module (`src/video_stabilization/`)

This module handles the reduction of camera shake in videos.

- **stabilize.py** : The `stabilize_video()` function is called by the Gradio UI. It orchestrates the process by initializing a `VidStabWrapper`, generating the transformations, applying them to the video, and generating plots if requested. It manages temporary files for the outputs.
- **vidstab.py** : A `VidStabWrapper` class simplifies the use of the external `vidstab` library. It encapsulates methods for generating and applying transforms, as well as for creating trajectory and transform plots.

5.4 Object Removal Module (`src/object_removal/`)

This module removes user-selected objects using inpainting.

- **inpainting.py** : The `remove_object()` function is the main entry point. It receives an image, a mask, and a method choice. It then calls the appropriate backend function: `cv2_inpainting()` or `deepfill_inpainting()`.
- **cv2_inpainting()** : Implements traditional inpainting using `cv2.inpaint()` with either `cv2.INPAINT_NS` (Navier-Stokes) or `cv2.INPAINT_TELEA`.
- **deepfill_inpainter.py** : A `DeepFillInpainter` class manages the DeepFill model. Its `__init__` method loads the pre-trained TensorFlow model and sets up the session. The `inpaint()` method preprocesses the image and mask, runs inference on the model, and post-processes the output.

- `model/model.py` : Contains the TensorFlow graph definition for the DeepFill generative inpainting model, specifying its U-Net-like encoder-decoder architecture.

Chapter 6: Algorithms

This chapter details the core algorithms and computational methodologies that power the three main functionalities of VidStyler: Neural Style Transfer, Video Stabilization, and Object Removal.

6.1 Neural Style Transfer (NST)

The Neural Style Transfer module is implemented using the optimization-based approach pioneered by Gatys et al. This method treats the generation of the stylized image as an optimization problem, iteratively refining an image to simultaneously match the content of one image and the style of another.

1. **Feature Extraction:** A pre-trained VGG16 Convolutional Neural Network (CNN), frozen to prevent its weights from updating, is used as a fixed feature extractor. Deeper layers capture high-level content, while shallower layers capture textures and colors.

2. **Content Representation and Loss:**

- The "content" of an image is represented by the feature map activations from a single, deeper layer (`relu3_3`).
- The content loss ($L_{content}$) is the Mean Squared Error (MSE) between the feature maps of the original *content image* and the *generated image* at this layer.

$$L_{content} = \frac{1}{2} \sum_{i,j} (F_{ij}^l - C_{ij}^l)^2$$

3. **Style Representation and Loss:**

- The "style" is captured by the correlations between feature responses, represented by the **Gram matrix**.
- The style loss (L_{style}) is the sum of the MSE between the Gram matrices of the *style image* and the *generated image*, computed across multiple layers (`relu1_2` , `relu2_2` , `relu3_3` , `relu4_3`).

$$E_l = \frac{1}{4N_l^2 M_l^2} \sum_{i,j} (G_{ij}^l - A_{ij}^l)^2$$

$$L_{style} = \sum_{l=0}^L w_l E_l$$

4. **Optimization Process:**

- The total loss is a weighted sum: $L_{total} = \alpha L_{content} + \beta L_{style}$, where α and β are user-controlled weights.
- The **L-BFGS optimizer** iteratively updates the pixels of the generated image (initialized from the content image) to minimize this total loss.

6.2 Video Stabilization

The video stabilization module utilizes the `vidstab` library, which implements a 2D feature-based stabilization pipeline.

1. Motion Estimation (Keypoint Tracking):

- The system identifies distinctive "keypoints" in each frame using an algorithm like GFTT, SIFT, or ORB.
- It tracks the movement of these keypoints between consecutive frames to estimate the inter-frame motion (translation, rotation, scaling), represented by an affine transformation matrix. This sequence of matrices forms the raw camera path.

2. Motion Smoothing:

- To remove jitter, a **moving average filter** is applied to the raw camera path. The user-controlled `smoothing_radius` defines the size of the filter's window. A larger radius produces a smoother path.

3. Frame Warping and Compensation:

- The algorithm calculates the corrective transformation required to move each original frame from its position on the raw path to its new position on the smoothed path.
- This transformation is applied to the frame via **warping**, which realigns it to match the stable trajectory.

4. Border Handling:

- Warping often creates empty areas at the frame borders. These are handled via user selection: filling with `black`, `reflecting` edge pixels, `replicating` edge pixels, or dynamically cropping the video (`auto` border size).

6.3 Object Removal (Inpainting)

The object removal module fills user-specified regions with visually plausible content.

1. **Mask Generation:** The user draws over an object in the `ImageEditor`. The system generates a binary mask by computing the difference between the original image and the image with drawings, followed by thresholding and dilation.

2. Inpainting Methods:

- **Method A: OpenCV Inpainting (Traditional)**

This approach uses fast, classical computer vision algorithms:

- **Navier-Stokes (`cv2.INPAINT_NS`):** Treats the image as a fluid and propagates pixel information from the mask's boundary inward.
- **Telea's Method (`cv2.INPAINT_TELEA`):** Uses a Fast Marching Method that considers image gradients, making it more effective for natural images.

- **Method B: DeepFill (Deep Learning)**

This method uses a pre-trained TensorFlow model with a deep **Convolutional Neural Network (CNN)**.

- The network has an encoder-decoder architecture trained to understand semantic context.
- Instead of just copying pixels, it **generates** new, context-aware content to fill the hole. It uses **gated convolutions** and **contextual attention** to handle irregular

shapes and borrow relevant textures from distant parts of the image, leading to highly realistic results.

Chapter 7: Testing and Results

This chapter discusses the testing methodologies employed for VidStyler and presents the qualitative results obtained from using its core functionalities.

7.1 Testing Approach

VidStyler was tested through a combination of functional testing and usability testing to ensure correctness, robustness, and ease of use.

- **Functional Testing:** Focused on verifying that each core feature and its options worked as intended. This involved testing with a wide variety of inputs (different images, videos, parameters) and checking for graceful error handling with invalid inputs.
- **Usability Testing:** Involved interacting with the Gradio interface to assess its clarity, ease of workflow, and the effectiveness of feedback mechanisms like status messages and progress indicators.

7.2 Results and Discussion

The functional and usability testing yielded positive qualitative results, validating the project's design and implementation.

Neural Style Transfer Results

The NST module successfully applied artistic styles to content images. The quality was highly dependent on the `style_weight`, `content_weight`, and `iterations`. The VGG16-based L-BFGS approach, while slow, proved capable of producing high-quality artistic effects.

Style Transfer

This tool applies the artistic style of one image to the content of another using neural style transfer techniques.

Upload both a content image (the image you want to stylize) and a style image (the image with the artistic style you want to apply).

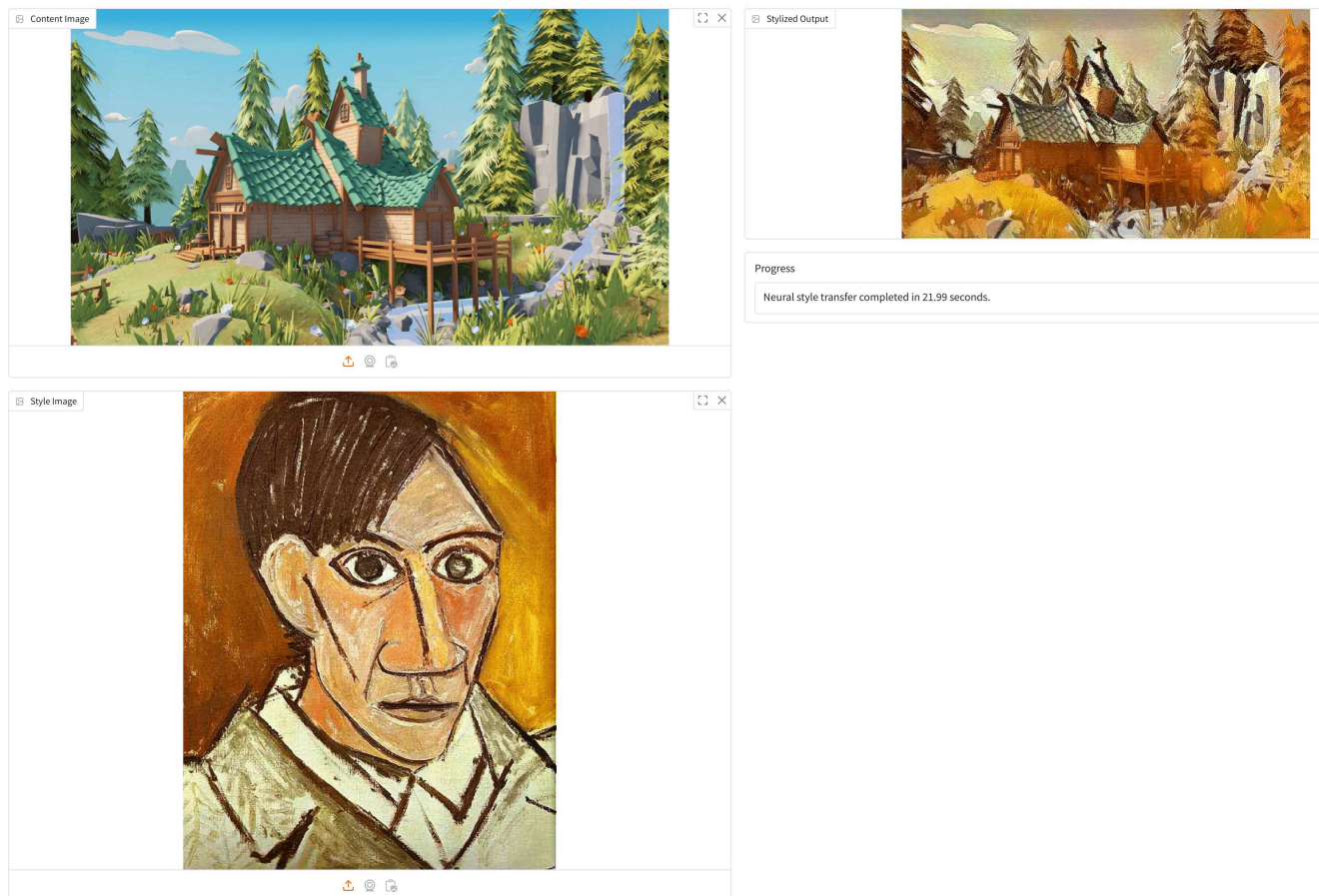


Figure 7.1: The UI for Neural Style Transfer, showing a content image (fantasy village) and a style image (abstract portrait) on the left. The right side displays the final output, where the village is rendered in the artistic style of the portrait.

Video Stabilization Results

The Video Stabilization module effectively reduced shakiness in test videos. The GFTT keypoint detector provided a good balance of speed and reliability. The `smoothing_radius` had a significant impact on the trade-off between smoothness and preserving intentional motion. The trajectory and transform plots were helpful for visualizing the stabilization process.

Video Stabilization

This tool reduces camera shake and unwanted motion in videos to create smoother footage.

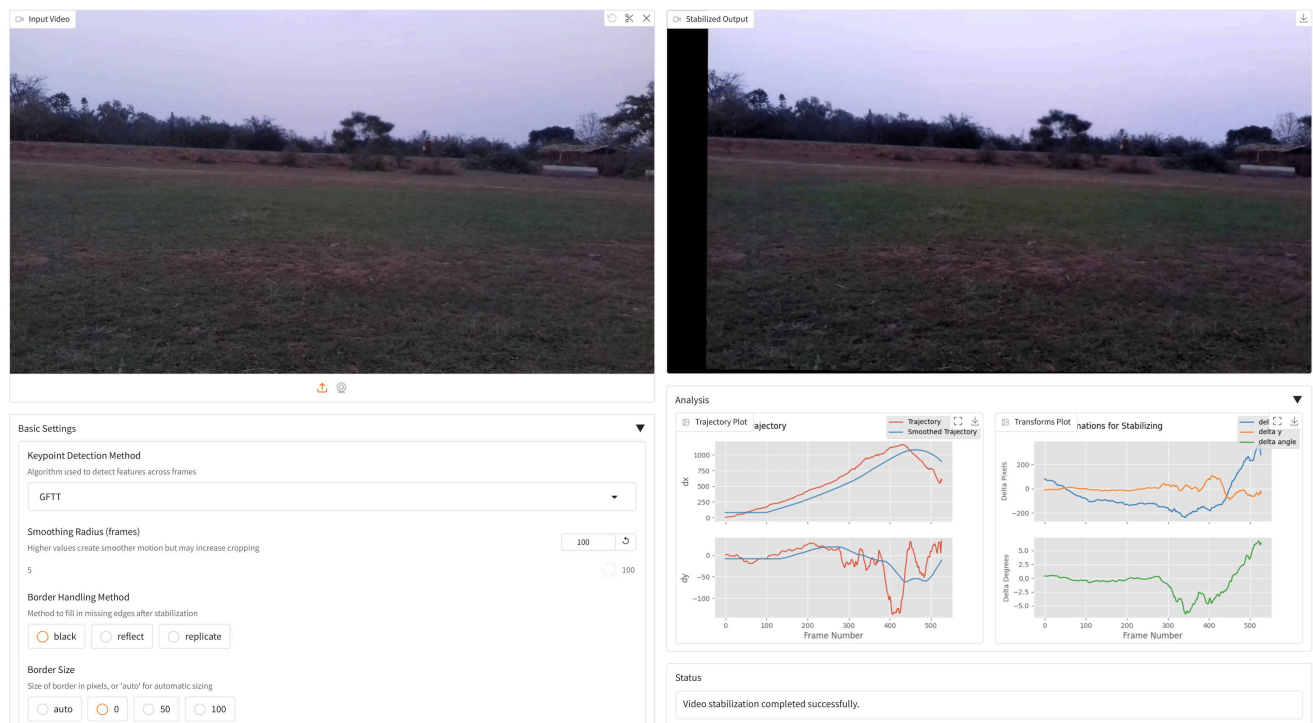


Figure 7.2: The Video Stabilization interface, with the original shaky video on the left and the stabilized output on the right. Below are the trajectory and transform plots, illustrating the correction applied to the camera's motion over time.

Object Removal Results

The Object Removal module successfully removed objects from images. A clear trade-off was observed between the two methods:

- **OpenCV Method:** Was very fast and worked well for small objects against simple backgrounds, but often resulted in blurring artifacts on complex scenes.
- **DeepFill (ML-based) Method:** Was significantly slower but produced much more plausible and contextually-aware results, especially for larger masked regions. It excelled at generating textures and structures that matched the surroundings.

Object Removal

This tool allows you to remove unwanted objects from images using intelligent inpainting technology. Draw over the objects you want to remove with the brush tool.

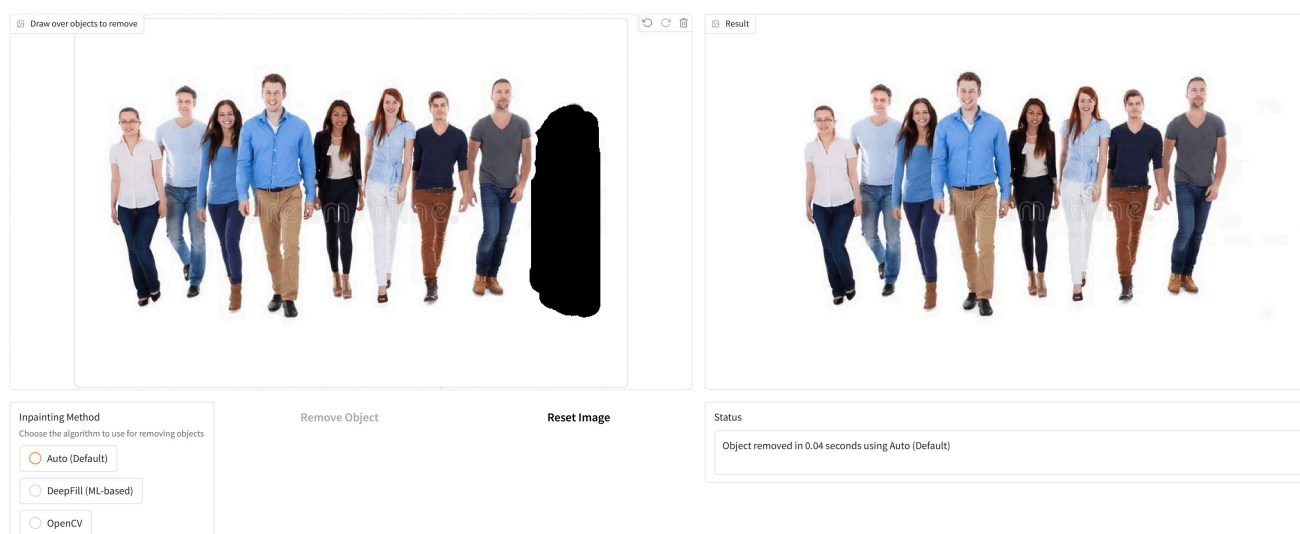


Figure 7.3: The Object Removal UI, where a mask has been drawn over a person in a group

photo. The result on the right shows the person seamlessly removed, with the background plausibly filled in by the DeepFill algorithm.

Performance Observations

- **Neural Style Transfer:** The most computationally intensive task. Processing a 512x512 image for 300 iterations took several minutes on a CPU but was significantly faster on a GPU.
- **Video Stabilization:** Performance scaled with video length and resolution. A short 720p video typically stabilized within a couple of minutes.
- **Object Removal:** The OpenCV method was near-instantaneous. The DeepFill method had a noticeable initial model loading time, followed by an inference time of a few seconds on a GPU.
- **UI Responsiveness:** The Gradio UI was generally responsive. During long backend processes, the UI would enter a waiting state, with status text boxes providing crucial feedback.

Chapter 8: Conclusion and Future Work

8.1 Conclusion

The VidStyler project successfully integrates three advanced AI-driven media editing functionalities—Neural Style Transfer (NST), Video Stabilization, and Object Removal—into a single, accessible application. This project's core achievement is its ability to bridge the gap between complex, state-of-the-art algorithms and a user-friendly interface. By leveraging powerful libraries such as PyTorch, TensorFlow, and OpenCV, and using Gradio to build an intuitive web interface, VidStyler democratizes access to sophisticated editing tools.

The NST module effectively produces high-quality artistic transformations, offering users precise control through adjustable parameters. The Video Stabilization feature proficiently reduces camera shake and is highly customizable. Furthermore, the Object Removal component provides a flexible solution by offering a trade-off between speed (via traditional OpenCV inpainting) and quality (via a context-aware DeepFill model).

The modular architecture of the application not only simplifies maintenance but also lays the groundwork for future expansion. The user-centric design of the Gradio interface abstracts the technical complexities, allowing users to achieve professional-looking results with minimal effort. VidStyler serves as a practical demonstration of how AI can empower a broader audience of content creators, making advanced media editing more approachable and efficient.

8.2 Future Work

While VidStyler meets its primary objectives, several avenues exist for further development to enhance its performance, features, and usability.

1. Performance Optimization:

- **Faster Style Transfer:** Integrate feed-forward style transfer models (e.g., TransformerNet, AdaIN) to offer a real-time alternative to the current L-BFGS optimization.
- **Inpainting Model Optimization:** Investigate newer, more optimized inpainting models or use model conversion techniques (e.g., ONNX) for faster inference.
- **Batch Processing:** Implement batch processing to allow users to process multiple images or videos in a single operation.

2. Feature Enhancements:

- **Video Functionality:** Extend Neural Style Transfer and Object Removal to video sequences, addressing the challenge of maintaining temporal consistency.
- **Automatic Mask Generation:** Integrate object detection models (e.g., YOLO, Mask R-CNN) to automatically generate masks for object removal.
- **Advanced Stabilization:** Explore real-time video stabilization previews and integrate a wider variety of state-of-the-art inpainting models.

3. UI/UX Improvements:

- **Interactive Feedback:** Implement more granular progress bars and provide more detailed status updates for long-running tasks.
- **Enhanced User Controls:** Add undo/redo functionality for the object removal mask and introduce parameter presets for common use cases.
- **Parameter Previews:** Develop a feature that allows users to see a quick, low-resolution preview of parameter changes before committing to a full process.

4. Model Management and Deployment:

- **Streamlined Setup:** Automate the downloading and setup of pre-trained models like DeepFill.
- **Containerization:** Use Docker to package the application for consistent deployment and easier dependency management.
- **Cloud Deployment:** Explore deploying VidStyler as a web service on cloud platforms to make it accessible without local setup.

By addressing these future directions, VidStyler can evolve into an even more robust, versatile, and user-friendly AI-powered media editing suite.

Chapter 9: References

Core Libraries & Frameworks:

- Abid, A., Abdalla, A., Ali, A., et al. (2019). Gradio: Hassle-Free Sharing and Testing of ML Models in the Wild. *arXiv preprint arXiv:1906.02569*.
- Paszke, A., Gross, S., Massa, F., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. *Advances in Neural Information Processing Systems*, 32.

- Abadi, M., Agarwal, A., Barham, P., et al. (2016). TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems. *arXiv preprint arXiv:1603.04467*.
- Bradski, G. (2000). The OpenCV Library. *Dr. Dobbs's Journal of Software Tools*.
- Heller, A. (2018). VidStab: Video Stabilization library for Python. GitHub repository. <https://github.com/AdamSpannbauer/vidstab>.

Key Algorithms & Research Papers:

- **Neural Style Transfer:**
 - Gatys, L. A., Ecker, A. S., & Bethge, M. (2015). A Neural Algorithm of Artistic Style. *arXiv preprint arXiv:1508.06576*.
 - Johnson, J., Alahi, A., & Fei-Fei, L. (2016). Perceptual Losses for Real-Time Style Transfer and Super-Resolution. *European conference on computer vision (ECCV)*.
 - Simonyan, K., & Zisserman, A. (2014). Very Deep Convolutional Networks for Large-Scale Image Recognition. *arXiv preprint arXiv:1409.1556* (VGGNet).
- **Object Removal (Inpainting):**
 - Yu, J., Lin, Z., Yang, J., Shen, X., Lu, X., & Huang, T. S. (2018). Generative Image Inpainting with Contextual Attention. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. (DeepFill v1)
 - Yu, J., Lin, Z., Yang, J., Shen, X., Lu, X., & Huang, T. S. (2019). Free-Form Image Inpainting with Gated Convolution. *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. (DeepFill v2)
 - Telea, A. (2004). An image inpainting technique based on the fast marching method. *Journal of graphics tools*, 9(1), 23-34. (OpenCV INPAINT_TELEA)
 - Bertalmio, M., Bertozzi, A. L., & Sapiro, G. (2001). Navier-stokes, fluid dynamics, and image and video inpainting. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. (Basis for OpenCV INPAINT_NS)
- **General Computer Vision & Machine Learning:**
 - Szeliski, R. (2010). *Computer Vision: Algorithms and Applications*. Springer Science & Business Media.
 - Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.